

JavaScript Deep Dive

Know the return shape

Choosing an array method starts with the shape you need back. One item, a new array and a yes-or-no answer are different results.

```
items.find(test);    // item or undefined  
items.filter(test);  // array  
items.some(test);    // boolean  
items.every(test);   // boolean
```

Find selects the first matching value. Filter creates an array containing matching values. Some asks whether a match exists; every asks whether all visited items pass.

A boolean cannot be followed by `.map()`. Check the return value before chaining methods.

Return is not decoration

With an arrow function, an expression body returns its expression. A block body needs an explicit return statement.

```
const a = [1, 2].map(n => n * 2);  
// [2, 4]
```

```
const b = [1, 2].map(n => { n * 2; });  
// [undefined, undefined]
```

The multiplication in the second example happens, but its value is not returned by the callback. Map collects the callback results.

Close the document and explain this bug out loud before checking the code again.

A closure keeps access

A closure is a function together with access to its surrounding lexical environment. The function can keep accessing that environment after the outer call has returned.

```
function counter() {  
  let value = 0;  
  return () => {  
    value += 1;  
    return value;  
  };  
}  
const next = counter();  
next(); // 1  
next(); // 2
```

Two separate calls to `counter()` create two independent environments.

Change a value, keep the original

To update an item without mutating the source array, create a new array and a new object for the item that changes.

```
const updated = games.map(game =>
  game.title === "Eclipse"
    ? { ...game, favorite: true }
    : game
);
```

Unchanged objects are reused here. This is a shallow update, not a deep clone of every nested object.

Primary reading: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures